Binary Logic: the computer's handling of 0s and 1s. The 0s and 1s are processed by **circuits**, which are electronic building blocks in a computer made from logic gates.

Circuits cost money and space. The more complex the circuit is, the more money and the more space it requires. We want a reliable way to take a circuit and find a simpler one that behaves the exact same way (cheaper, smaller, faster).

Boolean Algebra: set of math rules for working with 0s and 1s. It is the main tool used for circuit simplification. It allows us to manually simplify complicated logic expressions.

Binary Logic → Boolean Algebra → Simplified Circuits

Math systems (like Boolean Algebra) are built from three concepts:

1. Sets of elements:
   Set: a collection of objects.
   Elements: the objects inside a set.
   A set gives a math system structure. The easiest example is which numbers are restricted within the set.

2. Operators:
   Binary operator: rule that takes two inputs (two elements from a specific set) and produces a single element as the answer.

   For + to count as binary operator, two conditions must hold:
   1. It gives a clear definite rule for getting the answer c from the pair (a,b)
   2. Both the inputs (a,b) and the answer (c) must be in the same set.

   Ex. S = {1, 2, 3, 4, 5, …}
   2 + 3 = 5 + is a binary operator because the two inputs and the output are in the set
   2 - 3 = -1 - is NOT a binary operator because the output is NOT in the set.

3. Postulates (rules that we accept as true):
   These are the standard properties an operation might have. It doesn't necessarily mean that the operation has to show all of these:

   1. **Closure**: $x + y = z \quad x, y, z \in S$
      Combining any two elements in the set will always give an answer still in the set.

   2. **Associative**: $(x * y) * z = x * (y * z)$
      When you combine three things, it doesn't matter how you group them.

   3. **Commutative**: $x * y = y * x$
      Swapping the input order gives the same answer

4. **Identity**: there is a special element e, such that x * e = x
   e is the "do nothing" element. Combining it leaves the original unchanged.

5. **Inverse/Complement**: x * y = e
   Every element has an "undo" partner that cancels it back to its identity element.

6. **Distributive**: x * (y # z) = (x * y) # (x * z)
   You can "hand out" the outside operation to each piece inside the parenthesis.

Boolean Algebra, like any other math system, uses some of the postulates from above to derive its math system. These postulates are called the **Huntington Postulates.** They tell us how the AND/OR operators behave under the set $\{0, 1\}$.

Postulate	**(a) OR / $+$**	**(b) AND / $\cdot$**
1. Closure	closed under $+$	closed under $\cdot$
2. Identity	$x + 0 = x$	$x \cdot 1 = x$
3. Commutative	$x + y = y + x$	$xy = yx$
4. Distributive	$x + yz = (x + y)(x + z)$	$x(y + z) = xy + xz$
5. Complement	$x + x' = 1$	$x \cdot x' = 0$

Using the Huntington Postulates, we can confirm the validity of our three boolean algebra operators.

AND $(x \cdot y)$

x	y	$x \cdot y$
0	0	0
0	1	0
1	0	0
1	1	1

OR $(x + y)$

x	y	$x + y$
0	0	0
0	1	1
1	0	1
1	1	1

NOT (x')

x	x'
0	1
1	0

Closure: every entry in the tables is 0 or 1, so the answer never leaves the set.
Identities: 0 + 0 = 0 and 1 + 0 = 1 show 0 is the + identity
1 · 1 = 1 and 0 · 1 = 0 show 1 is the · identity
Commutative: each table is symmetric (the 01 and 10 rows match), so swapping inputs changes nothing.
Complement: from the NOT table, x + x′ = 1 and x · x′ = 0 hold for both x = 0 and x = 1.

Ex) Validate the following boolean expression using a truth table: x + yz = (x + y)(x + z):

x	y	z	yz	$x + yz$	$x + y$	$x + z$	$(x + y)(x + z)$
0	0	0	0	**0**	0	0	**0**
0	0	1	0	**0**	0	1	**0**
0	1	0	0	**0**	1	0	**0**
0	1	1	1	**1**	1	1	**1**
1	0	0	0	**1**	1	1	**1**
1	0	1	0	**1**	1	1	**1**
1	1	0	0	**1**	1	1	**1**
1	1	1	1	**1**	1	1	**1**